

Code-review response — clean/dirty calibration conventions: audit findings and the fix

For Liping · 2026-08-04 · fix commit `f7e9e7d` · full technical record: *WORKLOG 2026-08-04*

TL;DR

The question you raised in the code review was the right one to ask — and it did catch a real bug, just not the one hypothesized. The audit found that six of the seven engines (vanilla / vanilla-schedule / zero / FRN / fixed-then-float / ILB) were already calibrating clean-vs-clean and were self-consistent. The one outlier was the callable lattice — but it was **not** “dirty PV compared against clean BT”: it modelled **no accrued at all** on a snapped integer-period grid, and its genuine defects were in the grid *timing*. We rebuilt it to the exact convention (real coupon dates $\Rightarrow$ true dirty $\Rightarrow$ minus the shared accrued $\Rightarrow$ vs BT), and your invariance observation is now a permanent 16-test suite. Blast radius: 8 lattice-routed bonds moved by ≤ 35 bp with mixed signs; every other number in the book is bit-identical.

1. Your framework, stated mathematically

A model PV of future cash flows is intrinsically DIRTY (it discounts the next full coupon); the custodian BT is CLEAN (hard evidence: the file carries its own separate ‘Accrued income’ columns, and near-maturity high-grade names tie our clean price to BT within 0.02%). Accrued interest depends only on dates — never on the curve, the OAS, or any embedded option — so the two calibration forms

solve OAS s.t. $\text{dirty}(\text{OAS}) = \text{BT} + \text{AI}$	(dirty form)
solve OAS s.t. $\text{clean}(\text{OAS}) = \text{BT}$	(clean form, $\text{clean} \equiv \text{dirty} - \text{AI}$)

are the same equation with a constant added to both sides: **identical root**. That is your remark that “the implied OAS does not change with the price” — it is invariant to the price’s *representation* (clean vs dirty), not to its level. This identity is now a mechanically enforced test invariant (§4).

2. Audit result (every engine opened and read, not reasoned about)

Engine	Calibration target (as found in code)	Verdict
vanilla / vanilla-schedule	<code>implied_oas</code> solves <code>clean == BT</code> , <code>clean = dirty - AI</code>	✓ consistent
zero / STRIPS	<code>coupon = 0</code> $\Rightarrow$ <code>AI $\equiv$ 0</code> , <code>clean $\equiv$ dirty</code>	✓ trivially consistent
FRN	<code>clean == BT</code> ; <code>AI = coupon</code> fixed at the last reset $\times$ elapsed/364	✓
fixed-then-float	<code>clean == BT</code> ; <code>AI = fixed leg</code> accrued on the switch-anchored grid	✓
ILB	<code>clean == BT</code> ; <code>AI = real accrued $\times$ ratio₀</code> (matches the inflation-adjusted clean BT)	✓
price-to-call reference columns	via <code>price_bond</code> (clean form), not the lattice	✓
lattice	tree PV compared to BT directly — but the tree carries no accrued cash flow at all	⚠ see §3

Risk-metric denominators: `risk.py` / FRN / hybrid / ILB were already using the **dirty** (full) price, with the rationale documented in `risk.py`.

3. The lattice's actual problem — and why literally “subtracting AI” would have created a bug

The old lattice used a regular grid: `N = round(T·freq)` half-period steps, the valuation date treated as if it were a coupon date, the first coupon a full period away — **no stub, no accrued**. A PV built on that integer-period fiction is an *approximation of the clean price* (this fiction is precisely why clean quoting exists as a convention), so the calibration was clean-vs-clean **in intent** — the hypothesized systematic +AI overstatement was not there.

But the approximation only holds near par / near a coupon date, and the genuine defects were all timing ones:

1. the first coupon sat a full period out while the true stub is a fraction of one;
2. the maturity was snapped onto the half-period grid by `round`;
3. `T` and the call times were converted at **365.25 days/year** while the entire vanilla stack is **ACT/364** — two day counts fighting each other.

Concretely, for TNTD04441873 (6.45% of 2034, valued 2009-03-31): the real remaining schedule has **51** coupons (backward 182-day grid); the old tree carried **50** — the 365.25-vs-364 mismatch tipped the `round` and **dropped an entire coupon**.

Why the literal fix (“lattice PV – AI vs BT”) would have been wrong: the old PV already *approximately excludes* accrued through the integer-period fiction. Subtracting AI on top would double-count — the model price drops by $\sim$ AI, and the solved OAS is biased **low** by roughly 10–30 bp: it would have *manufactured* an error of exactly the predicted magnitude, with the opposite sign. Open-the-code-before-fixing — the `z_semi` lesson — applied once more.

The fix actually applied (`f7e9e7d`): the lattice grid is now the bond’s real ACT/364 coupon dates (variable-step BDT; the first step is the true stub), so the root PV is the **true dirty** price; calibration subtracts the shared vanilla accrued — `bond_price.accrued_interest`, the one formula in the codebase, which `price_bond` itself now consumes — and solves `clean == BT`. Call times convert at 364 d/y to match the grid. A strong new invariant falls out: **an option-free bond on the lattice reprices `price_bond`’s dirty price to machine precision**, i.e. the lattice and vanilla calibrators return the *same* OAS for the same target (tested).

4. Your remark, mechanized: `tests/test_price_convention.py` (16 tests; suite 145 green)

For every engine, the clean-form root (the engine’s own API) and an **independent** dirty-form root (a raw `brentq` on the engine’s dirty output plus the *shared* AI — deliberately not the engine’s own accrued) must agree to **< 1e-10**. Plus: shared-accrued identity locks for FRN/hybrid/ILB, `lattice` $\equiv$ `price_bond`, the valuation-date-on-a-coupon-date corner, and the zero-coupon AI = 0 case. Any future engine that mixes conventions — or grows a second private accrued formula — fails mechanically. Your review comment no longer depends on anyone remembering it.

5. Impact (the 8 lattice-routed bonds only; everything else verified bit-identical)

Bond	Implied OAS (bp) before → after	Eff-dur before → after
corp TNTD04115619 (BBB ‘13)	1959.0 → 1993.6 (+34.6)	3.42 → 3.31
corp TNTD04441873 (A ‘34)	412.3 → 410.8 (–1.5)	10.54 → 10.37 (straight 11.43, AQ 11.73)
corp TNTG701850W (EUR A ‘14)	293.2 → 305.6 (+12.4)	5.31 → 5.00
AGY FHLB 5.53 ‘14	160.5 → 190.0 (+29.5)	0.99 → 1.07 (AQ 0.87)

Bond	Implied OAS (bp) before → after	Eff-dur before → after
AGY FHLMC 5.30 '20	223.0 → 212.0 (−11.0)	4.30 → 3.72 (AQ 5.92)
AGY FHLMC 5.625 '35	181.2 → 178.9 (−2.3)	9.66 → 9.43 (AQ 9.74)
AGY FNMA 6.00 '36	194.1 → 191.4 (−2.6)	9.12 → 8.88 (AQ 9.62)
AGY FNMA 5.625 '21	197.0 → 189.7 (−7.3)	5.33 → 4.74 (AQ 5.38)

Note the changes are **mixed-sign and ≤ 35 bp**, not the uniform $\sim$ AI/duration downshift the dirty-vs-clean hypothesis would predict — which is itself evidence that the operative mechanism was grid timing, not an accrued mismatch. One honest caveat: the agency callables' fit to the custodian AQ loosened slightly (4/5 within 0.5y before, within 0.75y now). The old engine's two approximations (snapped grid + quasi-clean base) happened to offset in AQ's direction; we chose exact conventions and the machine-precision vanilla tie-out over an accidental fit to a custodian figure whose own model (vol, tree, OAS) is unknown.

6. The two side questions, settled

- **Duration denominator (dirty vs clean).** Computed both ways for all 61 bonds carrying a custodian AQ at 3-31: the dirty denominator is closer on 41/61, median $|\text{dur} - \text{AQ}|$ 0.236 vs 0.331 — and on the best-fitting informative subset (the TLGP block, errors 0.006–0.028) dirty wins consistently, so the custodian's AQ is itself full-price-based. **Dirty (full price) retained**, matching Bloomberg convention. Only the callable subset leans clean (4/5), but there the σ /par-call assumption noise is an order of magnitude above the 1–2% denominator effect — small-sample noise. Both variants are now standing columns in the driver outputs.
- **ILB accrued and the index ratio.** Confirmed consistent: accrued = real accrued $\times$ the valuation-date ratio_o, the same convention as BT (the inflation-adjusted clean, BT == BU/par·100); locked by a unit test.

7. If you want to verify it yourself

On 47: `.venv/bin/python -m pytest tests/test_price_convention.py -q` (16 passed; full suite 145). The refreshed `outputs/callable_risk.csv` and `outputs/phase2_risk_2009-03-31.csv` carry new `accrued` / `eff_dur_*cleanden` columns; the full before/after record is in `WORKLOG 2026-08-04` and `docs/headline_numbers_2026-08-04.md`.

Thank you for this review. The z_semi catch was two conflated bootstraps; this one was an engine with no explicit price convention — both were real convention-level findings, and both are now permanent tests. One more thing: item ④ on your 07-30 list, the AssuredGty 2066 call schedule (US04622DAA90), prices immediately on the fixed engine the day it arrives.